

9.b File Handling

LEVEL 1 — Foundations (but not trivial)

1. Log Analyzer

Input:

A log file with lines like:

```
1 2026-04-06 ERROR Disk full
```

Tasks:

- Count occurrences of each log level (`INFO` , `ERROR` , `WARNING`)
- Output summary to another file

Tip

Focus: parsing + reading line by line

2. CSV Student Processor

File Format:

```
1 name,marks
```

Tasks:

- Compute average
- Find topper
- Create a new file with grades (A/B/C)

Tip

Focus: string splitting + structured reading

3. Word Frequency Counter (Case-insensitive)

Input:

Large text file

Tasks:

- Count frequency of each word
- Ignore punctuation
- Output sorted result

 Tip

Focus: maps + cleaning input

LEVEL 2 — Intermediate (real thinking starts)

4. File Merge Tool

Input:

Two sorted files

Task:

- Merge into a third sorted file

 Tip

Think: merge step of merge sort, but with files

5. Persistent TODO Manager

Commands:

```
1 add Buy milk
2 list
3 done 2
```

Task:

- Store tasks in file permanently

Tip

Focus:

- file overwrite vs append
- maintaining state

6. Binary File Student Database

Struct:

```
1 struct Student {  
2     int id;  
3     char name[50];  
4     float marks;  
5 };
```

Tasks:

- Add record
- Search by ID
- Update marks

Important

Binary file I/O (`fstream` , `seekg` , `seekp`)

LEVEL 3 — Advanced (this is where skill forms)

7. External Sorting (Big Data Simulation)

Problem:

File too large to fit in memory

Tasks:

- Chunking
- Temporary files

- Merge

Tip

This is real-world file handling

8. File-Based Indexing System

Goal:

- Build index file:

word → file position

Task:

- Allow fast search without scanning whole file

Tip

Think: mini search engine

9. Log Rotation System

Condition:

- If file > X MB:
 - Rename old file → log_1.txt
 - Create new log file

Tip

Focus:

- file size
- rename
- filesystem operations



LEVEL 4 — High-Level / System Design

10. Mini Database Engine (File-based)

Commands:

```
1  INSERT id name marks
2  SELECT id
3  DELETE id
```

Task:

- Store everything in files

Important

Concepts:

- indexing
- record layout
- fragmentation

11. File Compression Tool (Basic)

Task:

- Implement Run-Length Encoding (RLE)

Tip

Learn:

- binary writing
- efficiency

12. Transaction System (Crash Safe)

Task:

- Before writing:

- log changes in file
- If crash → recover

Important

This is how databases actually work

Real Talk

Warning

Most people stop at:

- reading files
- writing files

That's useless.

If you finish **Level 3+**, you're ahead of:

| **90% of C++ devs in practical systems thinking**
